

Faculdade de Informática e Administração Paulista

Arthur Carvalho Brito Martins - RM 572325

Diego Soares Trujillo - RM 570147

Enzo Nukui da Silva - RM 569770

Leticia Cardoso de Almeida - RM 569415

Leticia Dias Araujo Felix Morato - RM
569138

LocalLead

Domain Driven Design using Java - Global Solution

São Paulo

2026

Sumário

1. Descritivo do Projeto	3
1.1 Explicação Detalhada	3
1.2 Justificativas e Objetivos	4
2. Objetivos dos Métodos	5
3. Sobre os Métodos	5
3.1 Método iniciarSistema()	6
3.2 Método consultarEstacao()	6
3.3 Método consultarClima()	6
3.4 Método mostrarPrevisao()	6
3.5 Método mostrarLotacao()	7
3.6 Método classificarLotacao()	7
3.7 Método mostrarIntervalo()	7
3.8 Método calcularIntervaloAtual()	7
3.9 Método mostrarStatus()	8
3.10 Método exibirMapa()	8
4. Demonstração Visual	9

1. Descritivo do Projeto

1.1 Explicação Detalhada

O LocalLead é um sistema desenvolvido com o objetivo de auxiliar usuários do transporte ferroviário urbano na consulta de informações relacionadas às linhas monitoradas pela solução. O projeto foi pensado para o contexto da mobilidade urbana na Região Metropolitana de São Paulo, com foco inicial nas linhas 11-Coral e 12-Safira da CPTM.

A solução permite que o usuário consulte dados sobre a linha escolhida, estação informada, clima, previsão, lotação estimada, status operacional e intervalo médio programado entre trens.

O sistema foi construído utilizando conceitos de orientação a objetos, separando as responsabilidades em diferentes classes para tornar o código mais organizado e coerente com o problema proposto.

A classe `LinhaCptm` representa a linha selecionada pelo usuário, armazenando informações como código, nome, cor e sentido. A classe `Estacao` representa a estação informada, relacionando-a com uma linha da CPTM.

A classe `Clima` interpreta as condições climáticas inseridas pelo usuário, enquanto a classe `Previsao` apresenta uma previsão estimada com base em regras simples do sistema. A classe `Lotacao` calcula uma estimativa de ocupação com base no horário atual, indicando se o período tende a ser de maior ou menor movimento. A classe `Intervalo` utiliza faixas horárias programadas para informar o intervalo médio entre os trens das linhas monitoradas.

Já a classe `StatusLinha` representa a situação operacional da linha, como operação normal, reduzida, paralisada ou encerrada. Além disso, a classe `InformacaoTransporte` funciona como uma classe base para informações que possuem fonte e data de atualização, sendo utilizada por classes como `StatusLinha` e `Intervalo`.

A classe `MapaTransporte` apresenta o recorte inicial do sistema, informando a região monitorada e as linhas analisadas.

Por fim, a classe `SistemaLocalLead` centraliza as principais consultas e conecta os objetos utilizados pelo sistema. Como o projeto Java não possui integração direta com APIs externas nesta etapa, algumas informações são inseridas pelo usuário por meio de caixas de diálogo, simulando uma entrada operacional. Outras informações são estimadas pelo próprio sistema, como a lotação por horário e os intervalos médios programados. Dessa forma, o LocalLead funciona como um protótipo acadêmico capaz de demonstrar como dados de transporte podem ser organizados e apresentados ao usuário.

1.2 Justificativas e Objetivos

A justificativa do projeto está relacionada à dificuldade enfrentada por muitos passageiros que dependem diariamente do transporte ferroviário urbano. Em grandes cidades, fatores como atrasos, superlotação, chuva, mudanças operacionais e longos intervalos entre trens podem impactar diretamente o tempo de deslocamento e o conforto dos usuários. Diante desse cenário, o LocalLead foi desenvolvido para reunir informações relevantes em um único sistema, permitindo que o usuário consulte dados básicos antes ou durante sua viagem. Mesmo sendo um protótipo, a proposta busca representar uma solução aplicável a um problema real de mobilidade urbana, utilizando tecnologia para apoiar a tomada de decisão do passageiro.

O projeto também se conecta ao tema da Global Solution ao propor uma solução tecnológica voltada para um desafio presente na Terra. A mobilidade urbana é um problema importante nas regiões metropolitanas, e sistemas inteligentes podem contribuir para tornar os deslocamentos mais previsíveis, acessíveis e organizados.

O principal objetivo do LocalLead é demonstrar, por meio da linguagem Java, como a orientação a objetos pode ser aplicada na construção de um sistema funcional. Para isso, foram criadas classes com responsabilidades específicas, representando elementos do domínio do transporte, como linha, estação, clima, previsão, lotação, status e intervalo.

Outro objetivo do projeto é aplicar conceitos estudados em aula, como criação de classes, atributos, métodos, construtores, encapsulamento, getters e setters, herança, estruturas condicionais, switch case, repetição com do while, tratamento de exceções com try/catch, uso de JOptionPane e manipulação de datas e horários com LocalDate e LocalTime.

O sistema também tem como objetivo simular uma experiência inicial de consulta para o usuário. A partir da linha escolhida, o programa permite acessar informações diferentes sobre a viagem, como a estação, o clima, a previsão, a lotação estimada, o status operacional e o intervalo médio entre os trens.

A utilização das linhas 11-Coral e 12-Safira foi definida como recorte inicial para manter o projeto mais objetivo e coerente com o escopo do protótipo. Dessa forma, o sistema não tenta representar toda a malha ferroviária, mas sim uma parte específica que pode ser expandida futuramente. Em uma versão futura, o LocalLead poderia ser integrado a APIs de transporte, dados climáticos reais, localização dos trens e informações em tempo real.

No entanto, para esta etapa, o foco está na construção de uma solução Java organizada, funcional e alinhada aos conceitos de orientação a objetos. Portanto, o LocalLead tem como objetivo unir a proposta da Global Solution com os conteúdos

desenvolvidos na disciplina de Java, criando um protótipo que representa um problema real, utiliza classes coerentes com o domínio escolhido e apresenta funcionalidades úteis ao usuário de forma simples, clara e objetiva.

2.0 Objetivos dos métodos

Os métodos desenvolvidos no sistema LocalLead têm como objetivo organizar as funcionalidades do projeto e permitir que cada classe tenha uma responsabilidade própria. Eles evitam que todas as ações fiquem concentradas na classe principal, deixando o código mais claro, dividido e coerente com a orientação a objetos.

A função dos métodos é transformar os dados informados pelo usuário ou definidos pelo sistema em informações úteis para a consulta da viagem. Por meio deles, o programa consegue apresentar dados sobre linha, estação, clima, previsão, lotação, status operacional, intervalo entre trens e mapa do recorte monitorado.

Além disso, os métodos aplicam as principais regras de negócio do projeto, como a estimativa de lotação por horário, o cálculo do intervalo médio entre trens e a análise das condições climáticas. Dessa forma, cada classe fica responsável por tratar as informações relacionadas ao seu próprio contexto.

Os métodos também contribuem para melhorar a interação com o usuário, pois retornam mensagens organizadas e formatadas para exibição no `JOptionPane`. Assim, o sistema apresenta os resultados de forma simples e compreensível.

Portanto, os métodos do projeto têm como objetivo estruturar o comportamento do sistema, aplicar regras de negócio e apresentar informações úteis ao usuário, tornando o protótipo Java funcional e alinhado à proposta do LocalLead.

3.0 Sobre os métodos

Nesta etapa, são apresentados dez métodos importantes implementados no sistema LocalLead. Os métodos escolhidos representam as principais funcionalidades do projeto e demonstram como as classes trabalham com regras de negócio, organização de informações e interação com o usuário. Não foram considerados construtores, getters ou setters, conforme solicitado na documentação da disciplina.

3.1 Método `iniciarSistema()`

O método `iniciarSistema()` pertence à classe `SistemaLocalLead` e tem como objetivo apresentar uma mensagem inicial ao usuário. Ele exibe o nome do

sistema, a versão utilizada e a cidade base do projeto, além de explicar brevemente a finalidade da solução.

Esse método é importante porque marca o início da execução do sistema e contextualiza o usuário sobre o funcionamento do LocalLead. Ele também reforça a proposta do projeto, mostrando que o sistema foi desenvolvido para auxiliar na consulta de informações relacionadas à mobilidade urbana.

3.2 Método consultarEstacao()

O método `consultarEstacao()` pertence à classe `SistemaLocalLead` e recebe um objeto da classe `Estacao`. Sua função é apresentar os dados da estação informada pelo usuário, como nome, município ou região e linha associada.

Esse método é importante porque conecta a classe principal do sistema com a classe `Estacao`, permitindo que as informações sejam exibidas de forma organizada. Ele também demonstra o uso de objetos como parâmetros, já que a consulta não trabalha apenas com textos soltos, mas com uma estação representada por uma classe própria.

3.3 Método consultarClima()

O método `consultarClima()` pertence à classe `SistemaLocalLead` e tem como objetivo apresentar ao usuário uma análise climática relacionada ao transporte. Ele utiliza os dados armazenados na classe `Clima`, como condição climática, temperatura e umidade estimada, além de considerar informações da classe `Previsao`.

A partir dessas informações, o método retorna uma mensagem com a situação climática, o impacto no transporte e uma recomendação para o usuário. Caso esteja chovendo, o sistema alerta sobre possíveis impactos no deslocamento, como aumento no tempo de espera e maior fluxo de passageiros nas estações.

3.4 Método mostrarPrevisao()

O método `mostrarPrevisao()` pertence à classe `Previsao` e tem como objetivo apresentar uma previsão estimada ao usuário. Ele utiliza a data atual para definir um período climático aproximado e exibir informações como temperatura mínima, temperatura máxima, temperatura média, chance de chuva e risco de chuva.

Esse método é importante porque simula uma previsão dentro do protótipo Java, mesmo sem integração com uma API meteorológica real. A funcionalidade demonstra como o sistema pode trabalhar com datas, regras condicionais e cálculos simples para gerar uma informação útil ao usuário.

3.5 Método mostrarLotacao()

O método `mostrarLotacao()` pertence à classe `Lotacao` e é responsável por apresentar a estimativa de ocupação no transporte com base no horário atual.

Antes de exibir as informações, ele utiliza a classificação da lotação para definir o percentual estimado e o nível de ocupação.

Esse método é importante porque ajuda o passageiro a entender se aquele horário pode ter maior fluxo de pessoas. A funcionalidade contribui para uma melhor organização do deslocamento, permitindo que o usuário avalie se o momento da viagem tende a ser mais confortável ou mais cheio.

3.6 Método classificarLotacao()

O método `classificarLotacao()` pertence à classe `Lotacao` e tem como objetivo definir o nível de lotação com base no horário atual. Ele analisa diferentes faixas de horário e atribui um percentual estimado e uma classificação para o período.

Esse método representa uma regra de negócio importante do projeto, pois concentra a lógica da lotação dentro da própria classe `Lotacao`. Dessa forma, a classe `Main` não precisa calcular a ocupação diretamente, deixando o sistema mais organizado e coerente com a orientação a objetos.

3.7 Método mostrarIntervalo()

O método `mostrarIntervalo()` pertence à classe `Intervalo` e tem como objetivo apresentar o intervalo médio programado entre os trens da linha selecionada. O sistema considera o horário atual e o código da linha escolhida pelo usuário, aplicando as faixas horárias de dias úteis para as linhas 11-Coral e 12-Safira.

Após calcular o intervalo correspondente, o método exibe uma mensagem com os dados da linha, o horário atual, o intervalo médio entre os trens e a fonte da informação. Essa funcionalidade é relevante porque permite ao usuário ter uma noção aproximada do tempo médio de espera, mesmo sem integração com dados em tempo real.

3.8 Método calcularIntervaloAtual()

O método `calcularIntervaloAtual()` pertence à classe `Intervalo` e é responsável por identificar qual intervalo médio deve ser utilizado de acordo com a linha e o horário atual. Ele verifica se a linha selecionada é a 11-Coral ou a 12-Safira e, a partir disso, aplica a faixa horária correspondente.

Esse método é importante porque representa uma das principais regras de negócio do projeto. Ele utiliza dados programados de operação dos trens para calcular uma estimativa de intervalo, deixando a classe `Intervalo` responsável por essa lógica e mantendo a classe `Main` mais simples.

3.9 Método `mostrarStatus()`

O método `mostrarStatus()` pertence à classe `StatusLinha` e é responsável por apresentar a situação operacional da linha escolhida. Ele exibe os dados da linha, a situação informada pelo usuário, a mensagem gerada pelo sistema, se a linha está ou não em operação e a fonte da informação.

Essa funcionalidade é importante porque representa uma das principais necessidades do passageiro: saber se a linha está funcionando corretamente antes de iniciar o deslocamento. Além disso, o método organiza várias informações em uma única resposta, facilitando a leitura pelo usuário.

3.10 Método `exibirMapa()`

O método `exibirMapa()` pertence à classe `MapaTransporte` e tem como objetivo apresentar o recorte inicial monitorado pelo `LocalLead`. Ele mostra a região analisada, a quantidade de linhas monitoradas, a quantidade de estações principais e as linhas utilizadas no protótipo.

Esse método é importante porque deixa claro o escopo do sistema. Como o projeto não representa toda a malha ferroviária, o mapa informa que o protótipo trabalha inicialmente com as linhas 11-Coral e 12-Safira, mantendo a solução mais objetiva e coerente com a proposta da entrega.

4.0 Demonstração Visual

